



Figure 1: TrimSlice H250—a very compact product



Figure 2: Chromium ported to ARM—ACID3



Figure 3: Firefox can be compiled for any architecture

```
real    2m31.303s
user    3m41.690s
sys     0m7.320s
```

Well, this system appears to be not very fast, compared to an x86 PC. However, the significant advantage is, the net-top, including the active WiFi module and HDD, consumes roughly 5.4-6 W. Isn't that great?

Next, let us check how fast Chromium and Firefox are with ACID3 and SunSpider tests. We can install them with *aptitude install chromium-browser* and *aptitude install firefox*. Then, let's begin with the ACID3 test. Chromium passed without any problems (Figure 2), although it's a relatively old version (10) in the repo. Next is Mozilla Firefox 9, which also passed the test without trouble (Figure 3). The well-designed internal JS engines of both are unbeatable. Now we should try a more sophisticated benchmark, where CPU performance is important—SunSpider, as shown in Figures 4 and 5. Chromium shows marginally better results than Firefox. The results may differ greatly from your desktop PC, but please consider the footprint and portability of the TrimSlice net-top! Don't expect an experience like on a powerful desktop like an Intel Dual Core. Remember, we're working on energy-efficient hardware, so there's a balance between speed and energy consumption. But don't worry—you can visit most websites. Moreover, the lack of (official) Flash support is definitely a plus—huge amounts of advertisements are thus blocked by the missing plug-in. However, what if one wants Flash?

Well, there's a little hope; we could re-use the Flash library from Android tablets. Officially, Flash is supported on ARM-based Androids, though it has no dedicated download package at the Adobe site at all. Why take a chance with the Android version? Simple—this is the same Linux!

```
$ wget http://kotelett.no/aci00/phh/Android2.2/libflashplayer.so
$ sudo cp libflashplayer.so /usr/lib/mozilla/plugins/
```

Let's run Firefox and, voila! There's Flash support on our TrimSlice unit now (Figure 6), despite the fact that it's Ubuntu Linux with Firefox, and not Android. Most important is stability—it's highly advisable to use Firefox instead of Chromium; Flash with the latter managed to kill the entire system. What's wrong is probably the V8 engine or general security, who knows!

Unfortunately, the Flash player cannot use the built-in Tegra2 hardware video acceleration, so it's rather uncomfortable to watch videos with higher resolutions than 480p. Why this is so, we will try to figure out in the next section.

Video acceleration in Tegra2

Despite the fact that broad support for video and audio formats has been declared to be hardware accelerated by the Nvidia Tegra2 core, in practice, the situation is very poor. First, the Nvidia programmers' efforts to ensure hardware decoding moved to Android land. I guess Nvidia is more interested in selling Android tablets based on their CPU than other computer boxes running Ubuntu, or other distros. As a result, some desktop and mobile products (the TrimSlice H250 net-top and the Toshiba AC100 smart-book) have no hardware acceleration in Linux at all. To reassure Toshiba users: though it comes with Android by default, it can be transformed into a normal Linux network.

Lately (December 2011), Nvidia announced that the SDK release for Tegra2 (L4T, Linux4Tegra) was out for free download—so we can expect hardware acceleration support in MPlayer/VLC/GStreamer soon. All I can say about hardware acceleration is that it really works well. The internally distributed Nvidia test application shows that 1080p video playback consumes just 25 per cent of CPU resources! With software decoding, there's no such luck!

Unfortunately, neither MPlayer, nor VLC supports hardware acceleration features (functions) hidden inside Tegra2 chip. That's why you can see such a slow playback for HD videos. If you compare Tegra2 with OMAP (another ARM CPU family, but from Cortex-A9 series), the latter will give you more software points and provide a better computational performance. For example, Tegra2 floating-point implementation uses vfpv3-d16 instructions, while OMAP processors can run full-featured 32-bit instructions. There are also no extensions like SIMD Neon. Again, this can be surely compensated when graphics acceleration comes to town. Even the functions already in Tegra2 are enough to make a breakthrough.